

Mutation test

VERIFICAÇÃO E VALIDAÇÃO DE SOFTWARE

Alunos: Emile Vargas Bordin e Natan De Aragão Muller

Prof: Júlio Machado

OBJETIVO

- O que é teste de mutação;
- Qual sua utilidade;
- Como funciona;
- Exemplo prático utilizando Java e PIT.



TESTES TRADICIONAIS

Nem sempre testes passando significam testes fortes.

TESTES TRADICIONAIS

```
24     @Test
25     void testeMultiplicar() {
26         Calculadora calc = new Calculadora();
27
28         assertEquals(1, calc.multiplicar(a: 1, b: 1), EPSILON);
29     }
```

Nem sempre testes passando significam testes fortes.

TESTES TRADICIONAIS

```
24  @Test
25  void testeMultiplicar() {
26      Calculadora calc = new Calculadora();
27
28      assertEquals(1, calc.multiplicar(a: 1, b: 1), EPSILON);
29  }
```



Esse teste passa, mas é fraco

Ele valida apenas um caso muito específico.

O QUE É TESTE DE MUTAÇÃO



O QUE É TESTE DE MUTAÇÃO

Código original

```
return a - b;
```

Código mutante

```
return a + b;
```

O QUE É TESTE DE MUTAÇÃO

→ Se os testes detectarem a alteração:

✓ Mutante morto

Se os testes não detectarem:

→ ✗ Mutante sobrevivente

COMO FUNCIONA O TESTE DE MUTAÇÃO



COMO FUNCIONA O TESTE DE MUTAÇÃO

Código Original



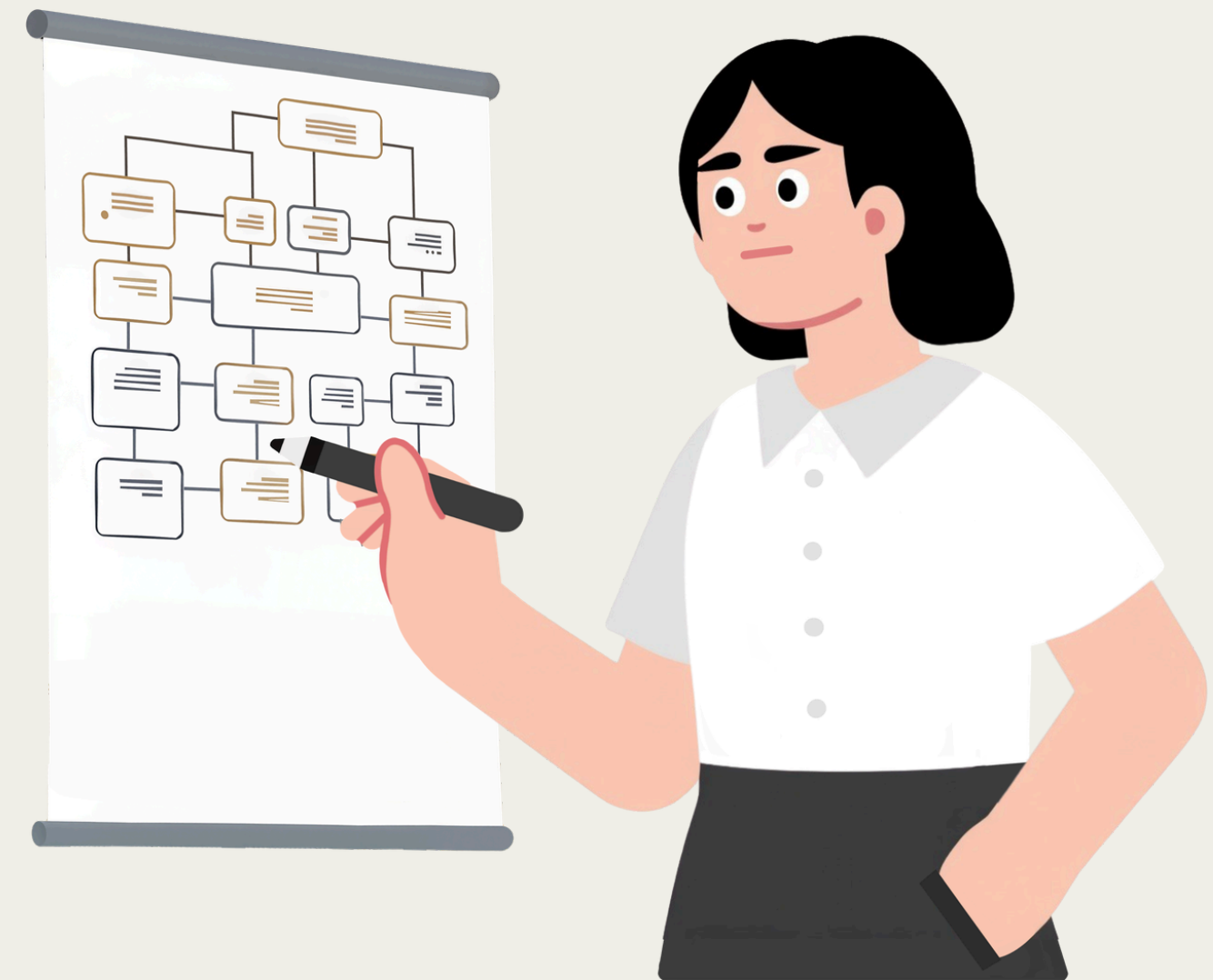
Geração de Mutantes



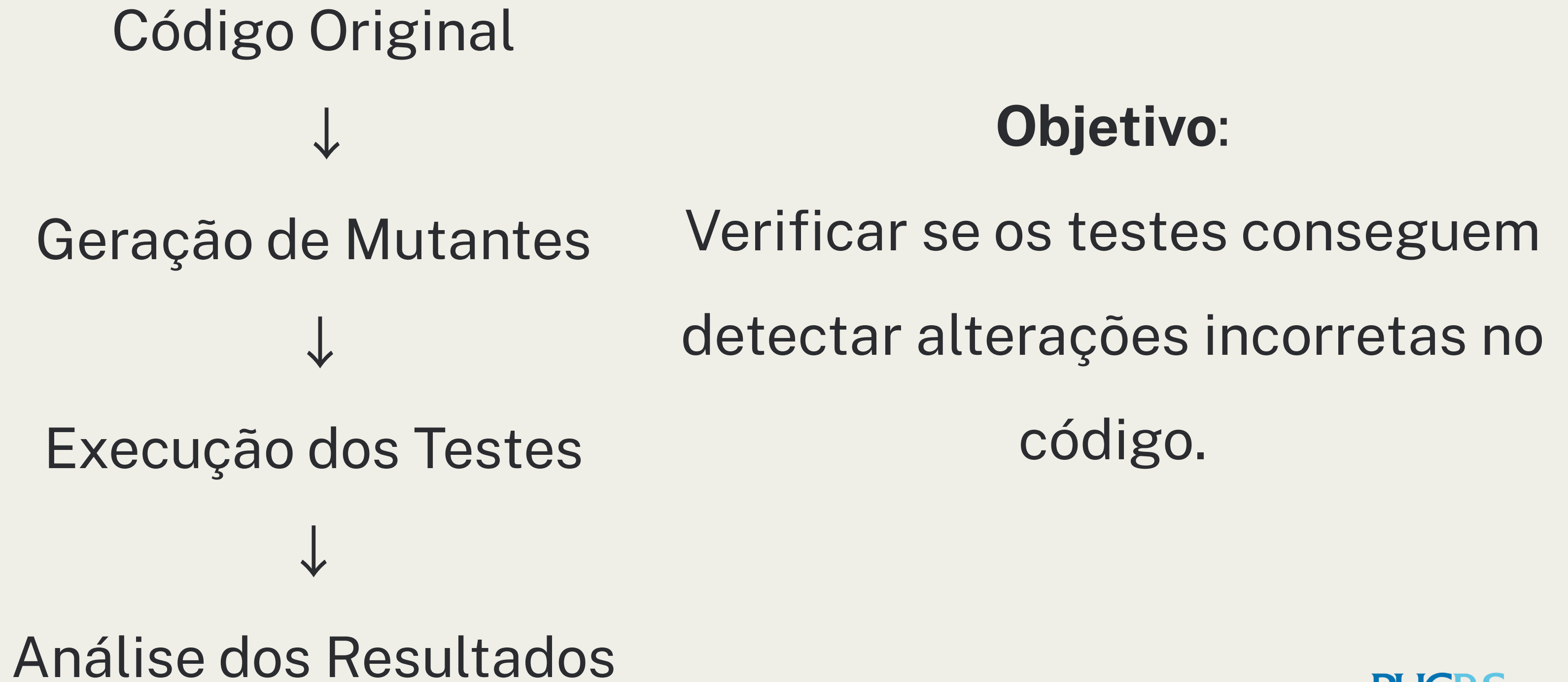
Execução dos Testes



Análise dos Resultados



COMO FUNCIONA O TESTE DE MUTAÇÃO



EXEMPLO DE MUTAÇÃO

```
return a * b;
```



```
return a / b;
```

UTILIDADE DO TESTE DE MUTAÇÃO

U

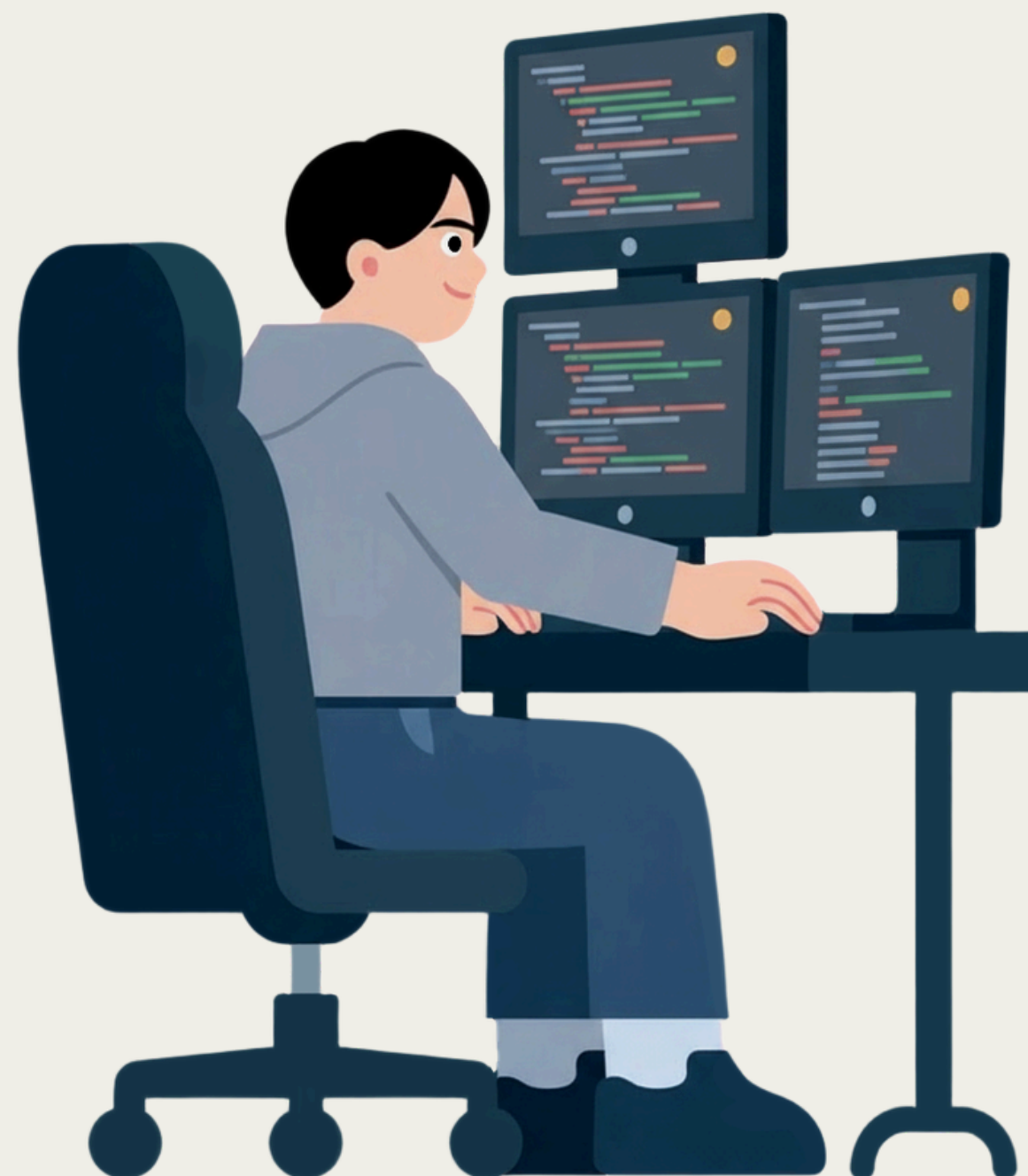
UTILIDADE DO TESTE DE MUTAÇÃO

- Identificar testes fracos;
- Melhorar a qualidade dos testes;
- Encontrar cenários não testados;
- Aumentar a confiabilidade do software;
- Avaliar melhor que apenas cobertura de código.

UTILIDADE DO TESTE DE MUTAÇÃO

Cobertura tradicional	Teste de Mutação
Verifica linhas executadas	Verifica força dos testes
Pode gerar falsa segurança	Detecta fragilidades reais
Mais superficial	Mais rigoroso

FERRAMENTAS UTILIZADAS



FERRAMENTAS UTILIZADAS

- Java

Linguagem utilizada no projeto.

- JUnit 5

Framework utilizado para criação dos testes automatizados.

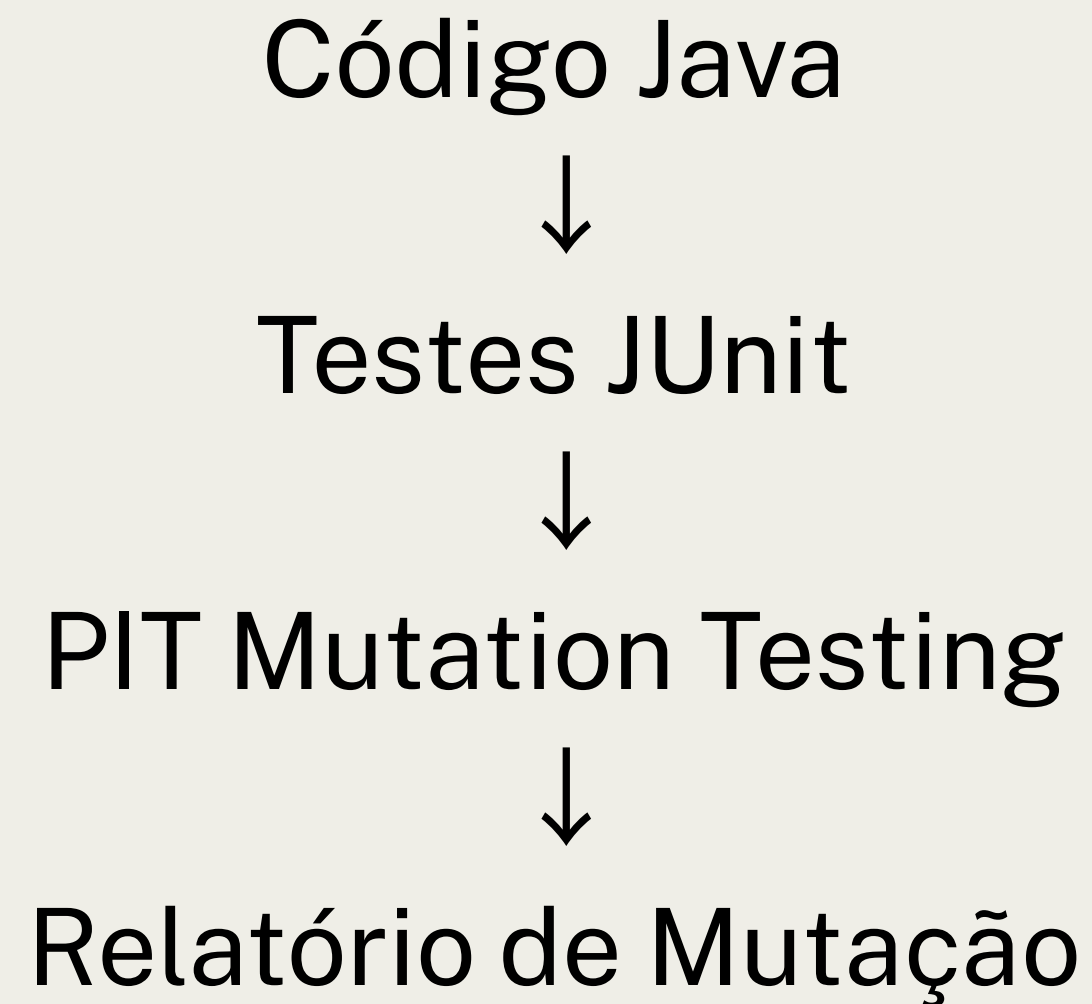
- Maven

Ferramenta utilizada para gerenciamento do projeto e dependências.

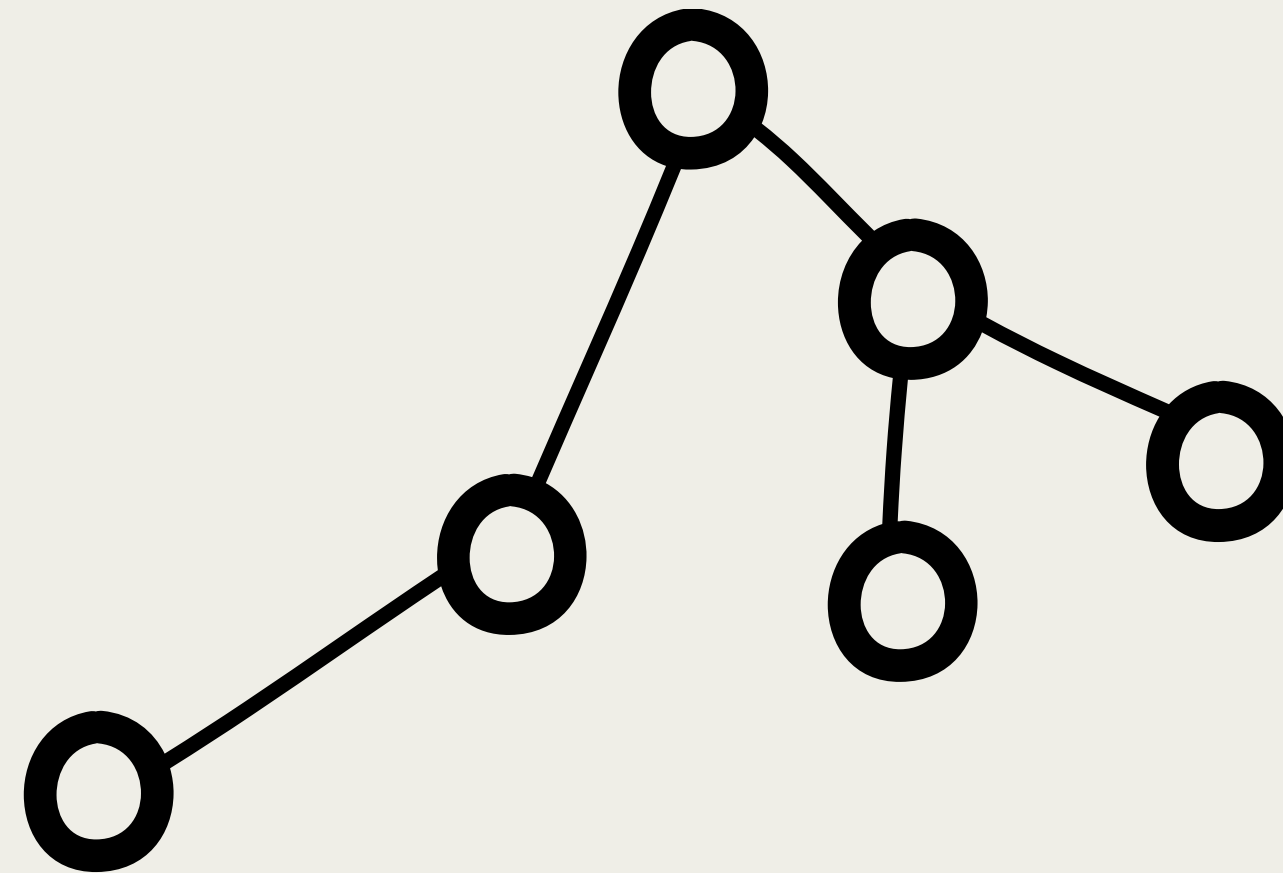
- PIT Mutation Testing

Ferramenta utilizada para geração e análise dos mutantes.

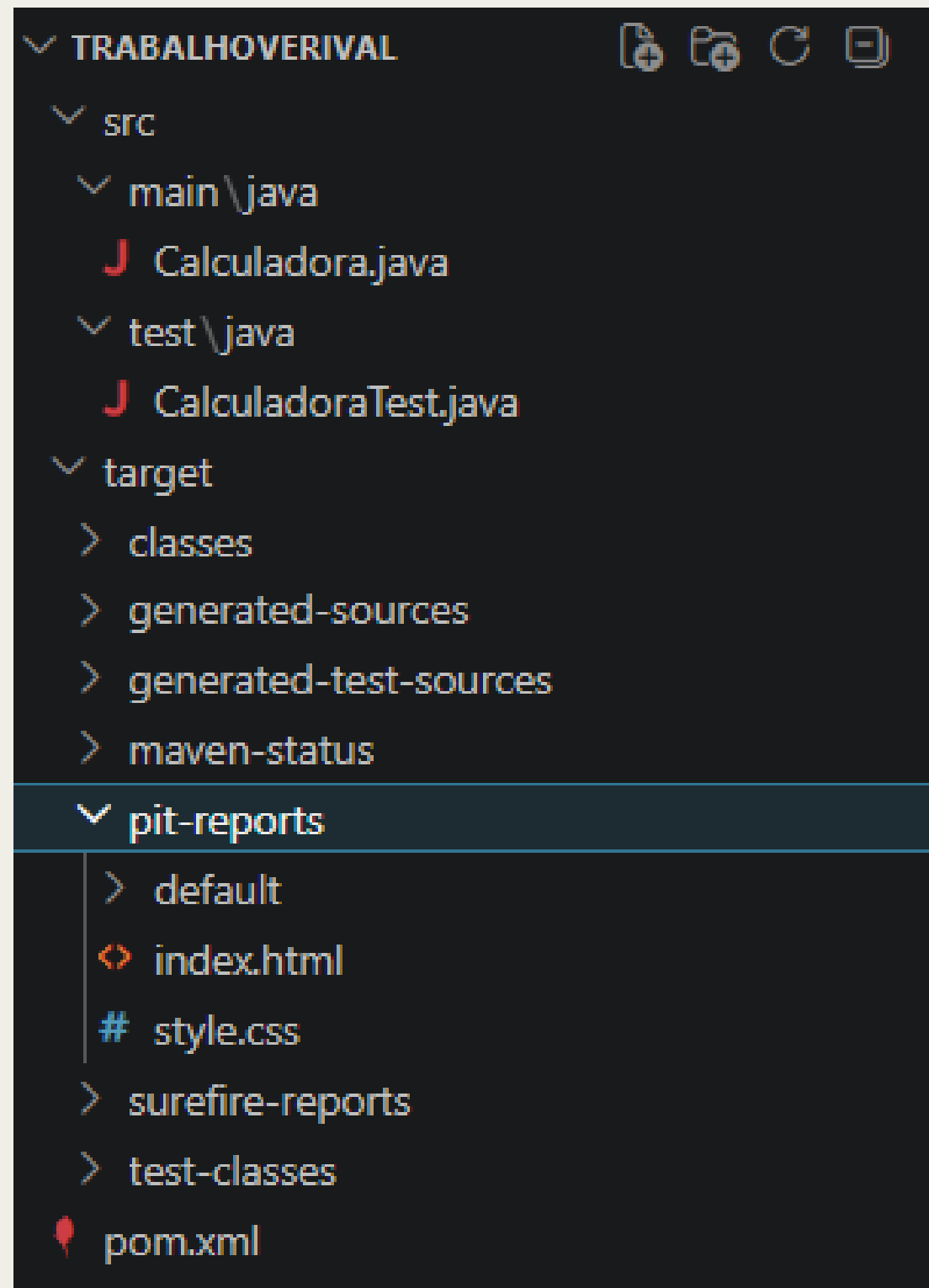
FERRAMENTAS UTILIZADAS



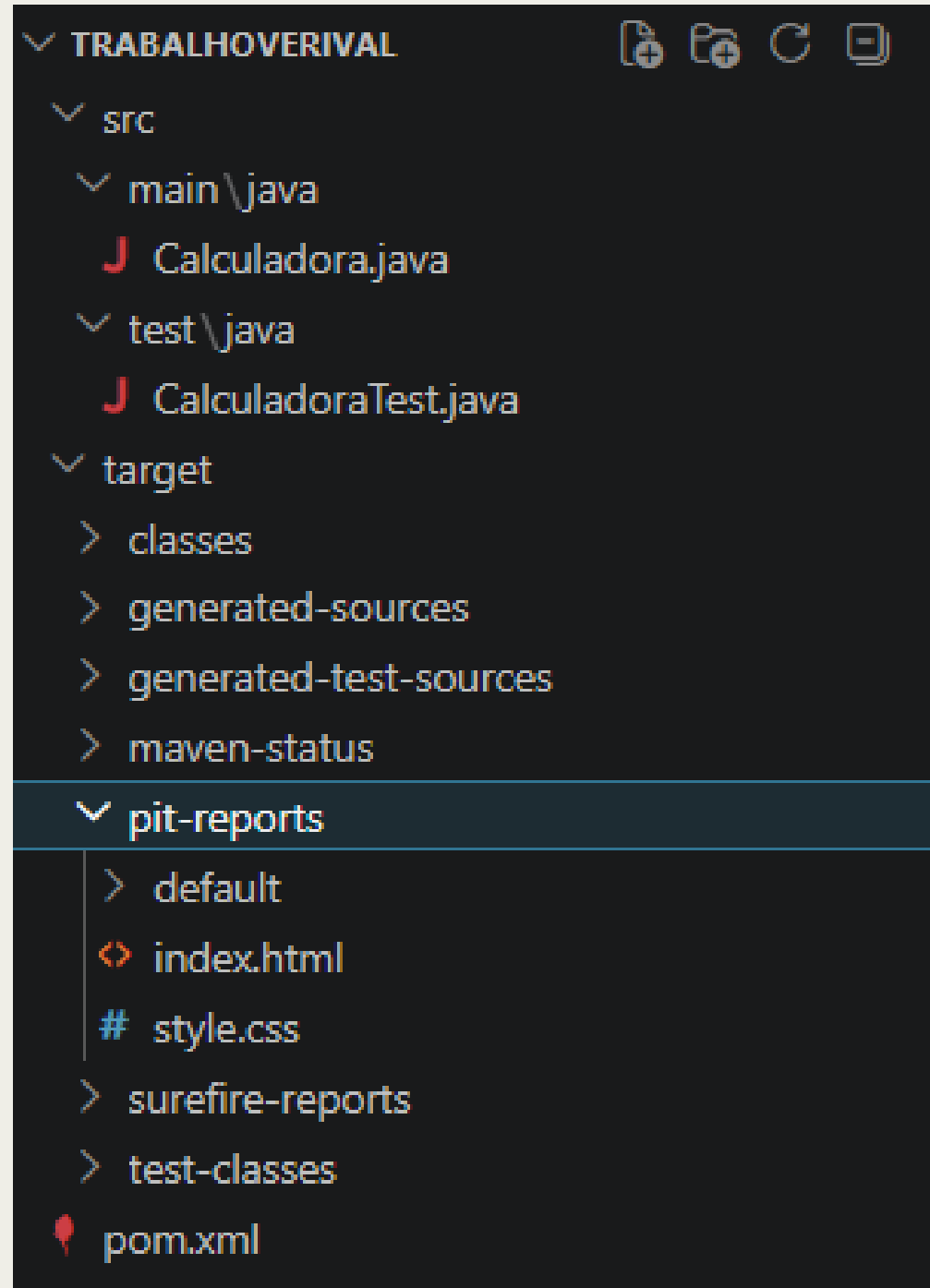
ESTRUTURA DO PROJETO



ESTRUTURA DO PROJETO



ESTRUTURA DO PROJETO



- Calculadora.java

Classe principal contendo as operações matemáticas.

- CalculadoraTest.java

Classe com os testes automatizados utilizando JUnit.

- pom.xml

Arquivo responsável pelas dependências e configuração do Maven e PIT.

- pit-reports

Relatórios gerados pelo teste de mutação.

Métodos presentes no projeto

- adicionar
- subtrair
- multiplicar
- dividir
- sen
- cos

CLASSE CALCULADORA

```
public class Calculadora {  
    public double adicionar(double a, double b) {  
        return a + b;  
    }  
  
    public double subtrair(double a, double b) {  
        return a - b;  
    }  
  
    public double multiplicar(double a, double b) {  
        return a * b;  
    }  
  
    public double dividir(double a, double b) {  
        if (b == 0) {  
            throw new IllegalArgumentException(s: "Divisão por zero");  
        }  
  
        return a / b;  
    }  
  
    public double sen(double x) {  
        return Math.sin(x);  
    }  
  
    public double cos(double x) {  
        return Math.cos(x);  
    }  
}
```

Classe responsável pelas operações matemáticas utilizadas nos testes.

TESTES AUTOMATIZADOS



TESTES AUTOMATIZADOS

O teste verifica:

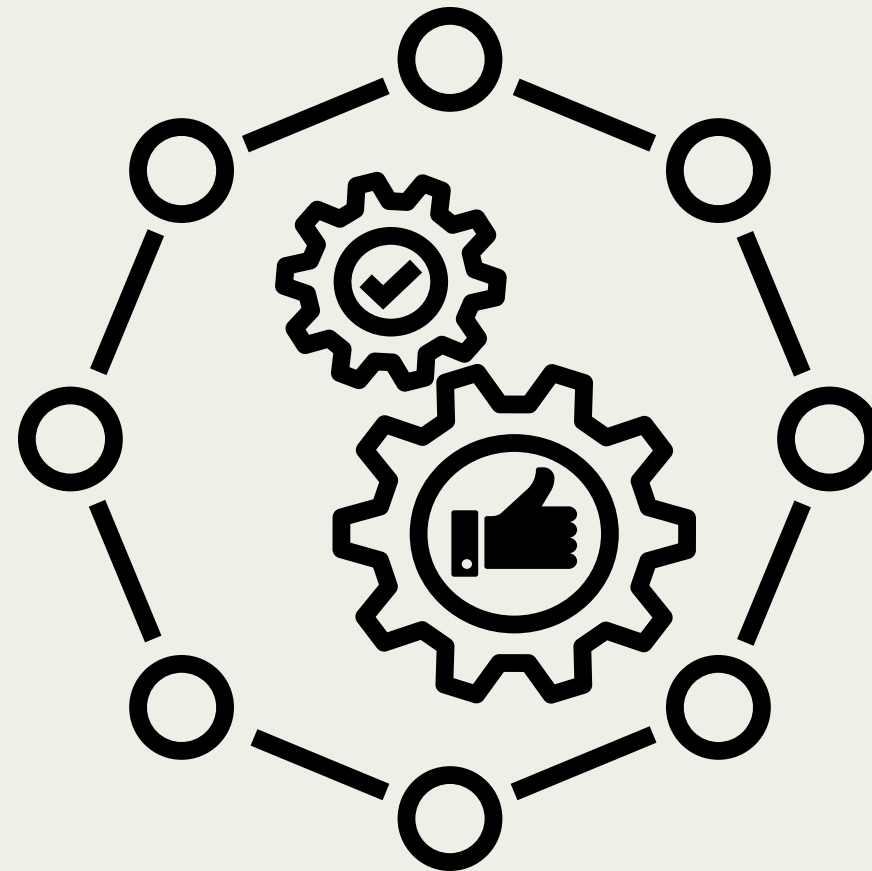
- ✓ Entrada utilizada
- ✓ Resultado esperado
- ✓ Funcionamento correto do método

```
@Test
void testeMultiplicar() {
    Calculadora calc = new Calculadora();

    assertEquals(6.0, calc.multiplicar(a: 2, b: 3), EPSILON);
}
```

Os testes automatizados foram desenvolvidos utilizando JUnit 5.

EXECUÇÃO DOS TESTES TRADICIONAIS



EXECUÇÃO DOS TESTES TRADICIONAIS

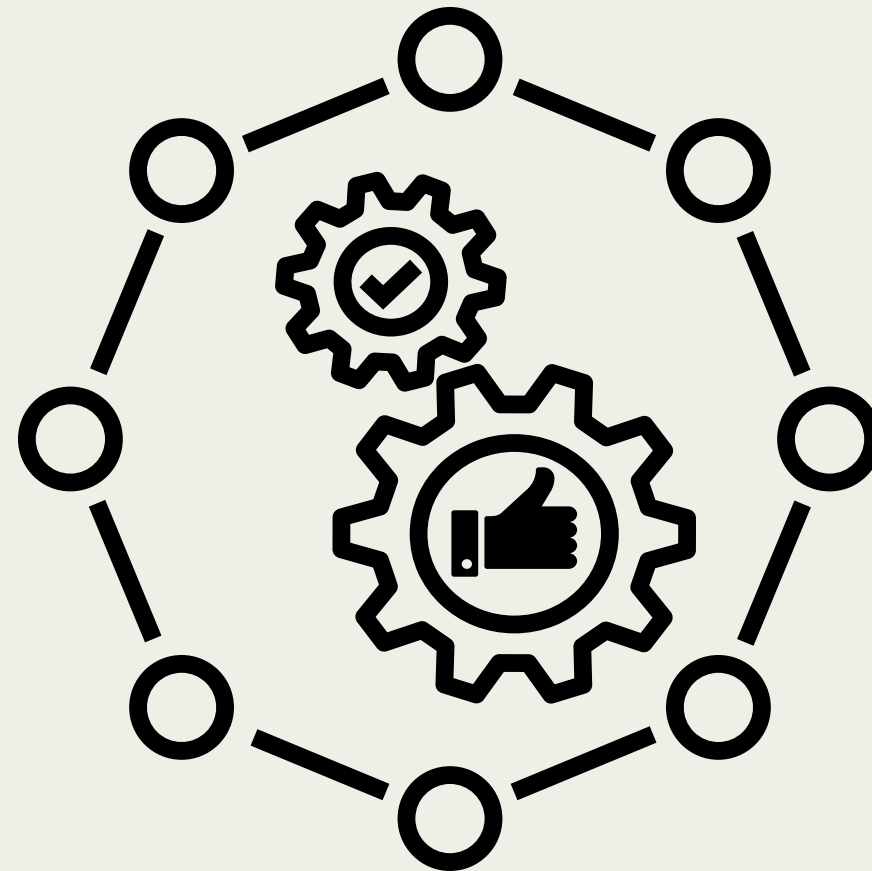
Comando utilizado

```
● PS C:\Users\emile\TrabalhoVerival> mvn test
```

Resultado Esperado

```
[INFO] -----  
[INFO] T E S T S  
[INFO] -----  
[INFO] Running CalculadoraTest  
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.081 s -- in CalculadoraTest  
[INFO]  
[INFO] Results:  
[INFO]  
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  
[INFO]  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 3.558 s  
[INFO] Finished at: 2026-05-11T18:25:05-03:00  
[INFO] -----
```

EXECUÇÃO DOS TESTES DE MUTAÇÃO



EXECUÇÃO DOS TESTES DE MUTAÇÃO

Comando utilizado

```
PS C:\Users\emile\TrabalhoVerival> mvn org.pitest:pitest-maven:mutationCoverage
```

Resultado Esperado

```
[INFO] -----  
[INFO] T E S T S  
[INFO] -----  
[INFO] Running CalculadoraTest  
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.081 s -- in CalculadoraTest  
[INFO]  
[INFO] Results:  
[INFO]  
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0  
[INFO]  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 3.558 s  
[INFO] Finished at: 2026-05-11T18:25:05-03:00  
[INFO] -----
```

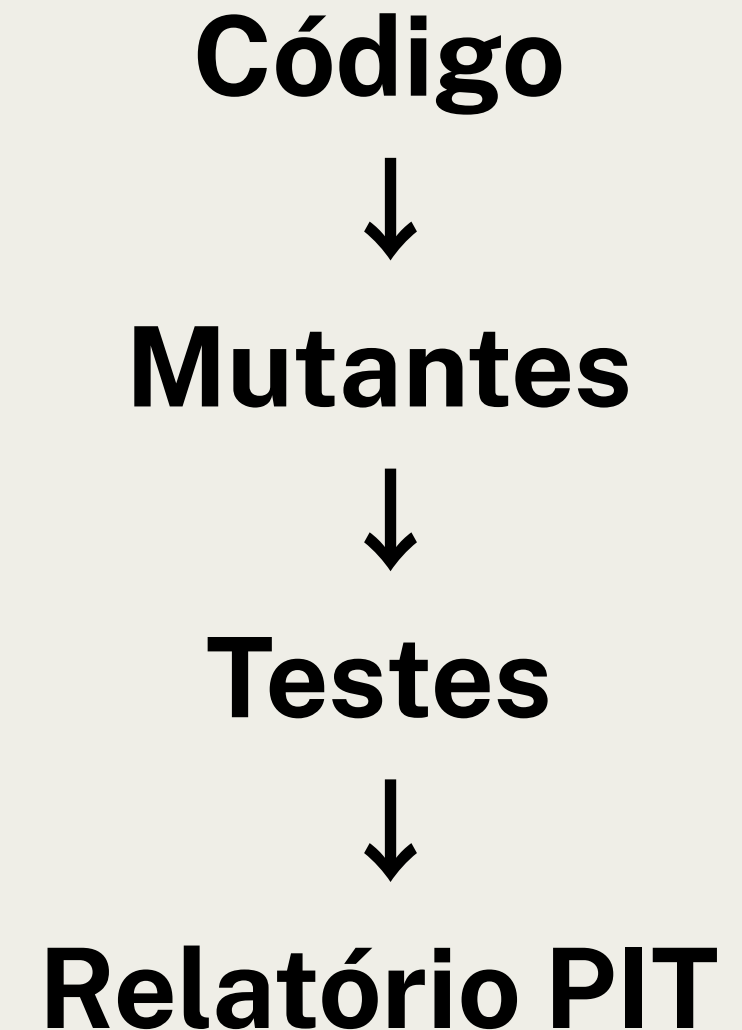
EXECUÇÃO DOS TESTES DE MUTAÇÃO

PROCESSO REALIZADO PELO PIT

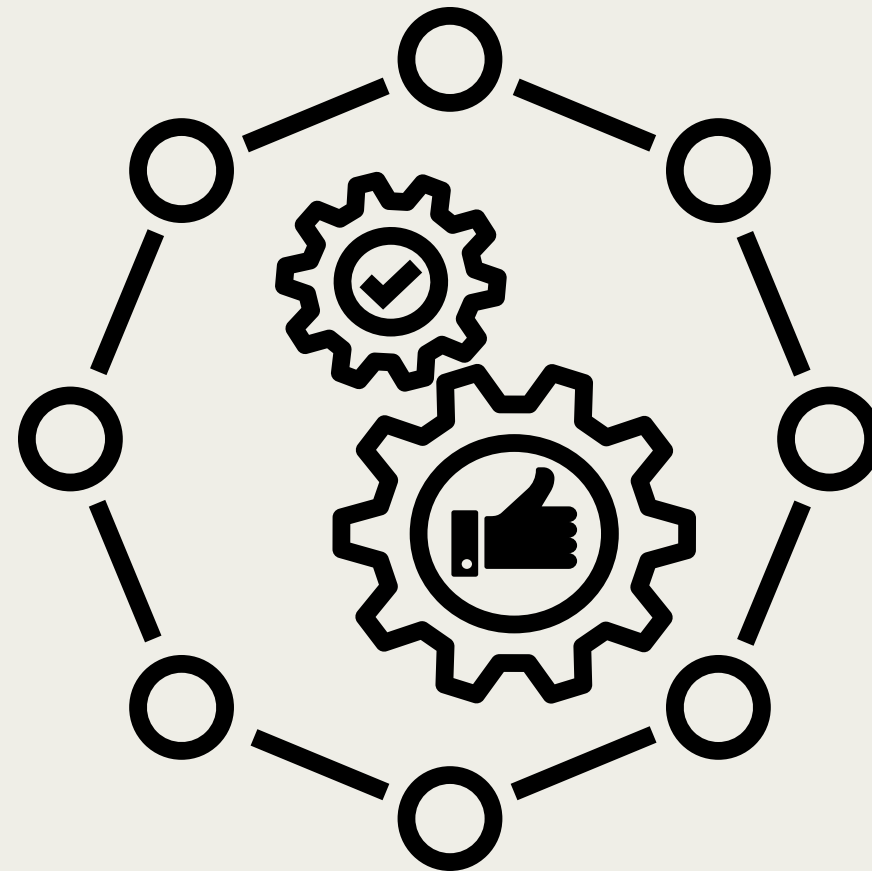
- Geração automática de mutantes;
- Execução dos testes sobre os mutantes;
- Identificação de mutantes mortos e sobreviventes;
- Geração de relatório HTML

CAMINHO DO RELATÓRIO

<target/pit-reports/index.html>



EXEMPLOS DE MUTAÇÕES GERADAS PELO PIT



EXEMPLOS DE MUTAÇÕES GERADAS PELO PIT

- Addition → Subtraction
- Multiplication → Division
- Negated Conditional
- Return → 0.0

As mutações são geradas automaticamente pela ferramenta.

EXEMPLO PRÁTICO



VANTAGENS E LIMITAÇÕES



VANTAGENS E LIMITAÇÕES

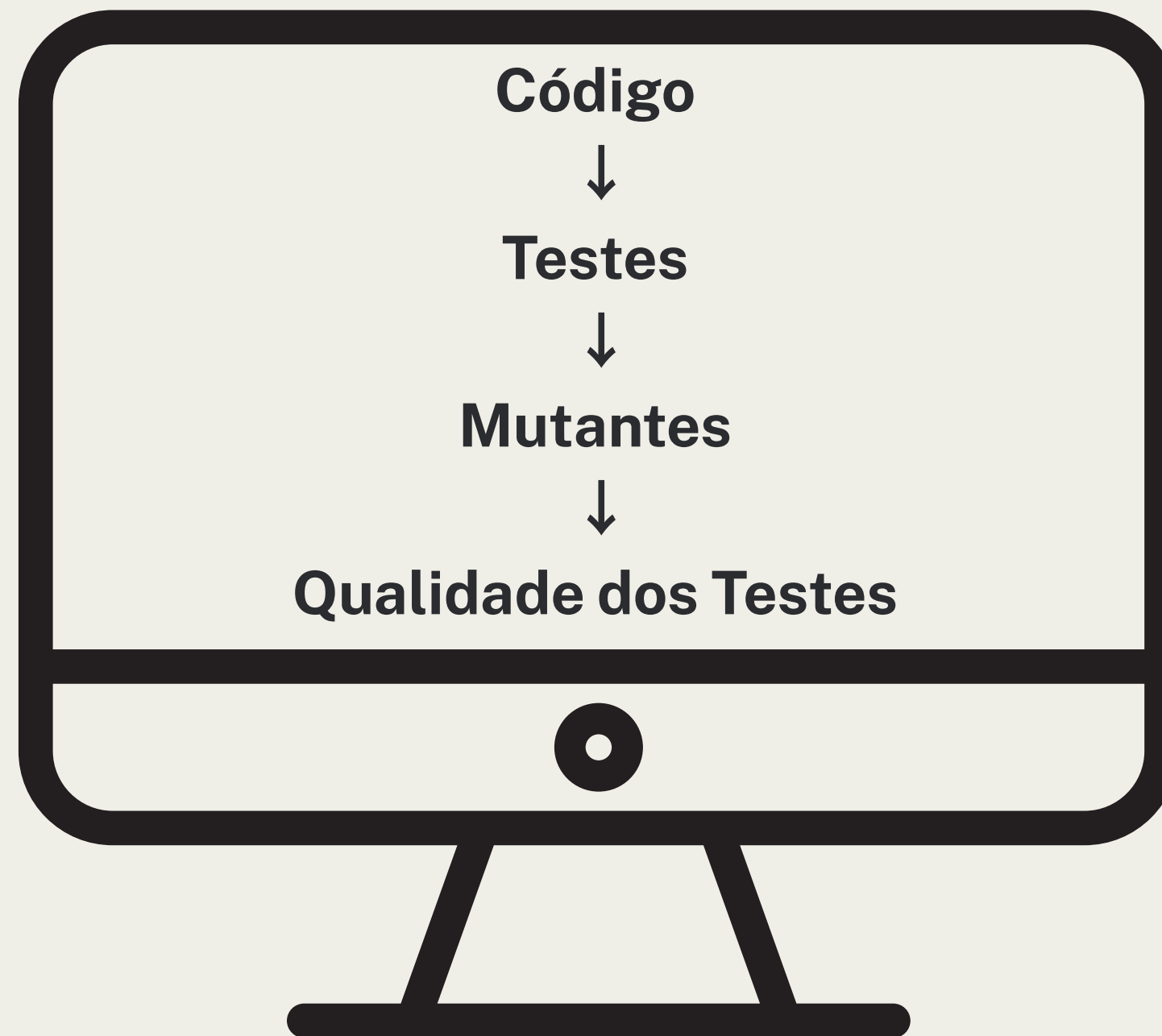
Vantagens

- ✓ Avalia qualidade real dos testes
- ✓ Identifica testes fracos
- ✓ Melhora confiabilidade do software
- ✓ Complementa cobertura de código
- ✓ Ajuda a criar testes mais robustos

LIMITAÇÕES

- ⚠ Pode aumentar tempo de execução
- ⚠ Gera muitos mutantes
- ⚠ Alguns mutantes podem ser equivalentes
- ⚠ Exige análise dos resultados

CONCLUSÃO



O objetivo do teste de mutação não é testar o sistema, mas sim testar os próprios testes.

REFERÊNCIAS

[1] APACHE MAVEN PROJECT. Maven. [S. l.], 2026. Disponível em: <https://maven.apache.org/>. Acesso em: 7 maio 2026.

[2] JUNIT. JUnit 5 User Guide. [S. l.], 2026. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 8 maio 2026.

[3] ORACLE. Java Documentation. [S. l.], 2026. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 10 maio 2026.

[4] PITEST. PIT Mutation Testing. [S. l.], 2026. Disponível em: <https://pitest.org/>. Acesso em: 10 maio 2026.

[5] BAELDUNG. Mutation Testing with PIT. [S. l.], 2026. Disponível em: <https://www.baeldung.com/java-mutation-testing-with-pitest>. Acesso em: 8 maio 2026.

[6] GEEKSFORGEEKS. Mutation Testing - Software Engineering. [S. l.], 2025. Disponível em: <https://www.geeksforgeeks.org/software-engineering/software-testing-mutation-testing/>. Acesso em: 7 maio 2026.

[7] PITEST. Quick Start Guide. [S. l.], 2026. Disponível em: <https://pitest.org/quickstart/>. Acesso em: 9 maio 2026.